

Recovering the Zipfian Distribution in Unsupervised Term Discovery

Danel Slabbert, Simon Malan, Herman Kamper

Electrical and Electronic Engineering, Stellenbosch University, South Africa

24051055@sun.ac.za, 24227013@sun.ac.za, kamperh@sun.ac.za

Abstract—Unsupervised term discovery involves segmenting unlabelled speech into word- or syllable-like units and clustering these into a lexicon of candidate types. True lexicons follow a Zipfian distribution, yet the dominant centre-based clustering approach—K-means—produces a more uniform distribution due to an inductive bias toward spherical clusters. In this paper we revisit graph-based clustering as a bottom-up alternative, where segment embeddings are connected by pairwise similarity and partitioned using the Leiden algorithm. We show that graph clustering substantially outperforms centre-based approaches (K-means, GMM, BIRCH) in both word- and syllable-level lexicon discovery across three languages, producing more Zipf-like distributions. Another bottom-up approach, agglomerative clustering with average linkage, also performs well, although it is computationally less efficient and allows for less control over the resulting distribution. Our work calls into question the dominance of centre-based clustering for term discovery, and promotes graph clustering as an attractive alternative.

Index Terms—word segmentation, word discovery, lexicon learning, zero-resource speech processing, unsupervised learning

I. INTRODUCTION

Unsupervised term discovery builds a lexicon from unlabelled speech by identifying boundaries and then clustering the resulting segments into phonetically homogeneous groups [1]. Although human infants achieve this within their first year of life [2]–[4], it remains a challenging problem for computational systems. The difficulty stems from the absence of explicit word delimiters and the acoustic variability across instances of the same word [5]. Developing models that approximate this ability can inform theories of language acquisition [6] and support speech technologies in low-resource settings [7].

Early approaches for term discovery identified recurring acoustic patterns using dynamic time warping and clustered them via a similarity graph [8], [9]. However, they relied on conventional acoustic features that capture surface properties of the signal. Self-supervised learning (SSL) changed this: models trained on large quantities of unlabelled speech learn representations that are far more robust to speaker and acoustic variability while encoding richer phonetic structure [10], [11]. Modern discovery systems therefore segment and cluster speech in learned embedding spaces [12]–[21].

Utilising SSL features, most discovery systems now first segment speech before clustering the resulting segments. For clustering, K-means has seen widespread adoption due to its simplicity and efficiency [22], [23]. However, K-means has an

inductive bias toward similarly sized clusters. As illustrated in Fig. 1, the resulting K-means distribution is therefore poorly aligned with the long-tailed Zipfian type-frequency distributions characteristic of natural language [24]. Other common clustering approaches [25]–[27] also favour compact clusters around representative centres.

This paper investigates whether we can recover the Zipfian distribution in the lexicons of unsupervised term discovery systems by using a clustering method with a different inductive bias. To this end, we compare three centre-based clustering methods (K-means, BIRCH, GMMs) with two bottom-up alternatives (graph clustering, agglomerative clustering). Graph clustering—which was used in the earliest term discovery work [8]—is particularly attractive because its hyperparameters provide control over the properties of the resulting lexicon. In addition, we evaluate the five clustering methods under different segmentation granularities to assess how this affects lexicon induction: ground-truth word boundaries, providing an upper bound; ground-truth syllable boundaries, which evaluate clustering over shorter, more uniform units; and an unsupervised syllabic segmentation, reflecting a fully unsupervised setting with noisy segments. We are particularly interested in the syllabic units because these can generally be segmented more reliably than words by unsupervised systems [13], and because they have seen growing adoption in spoken language

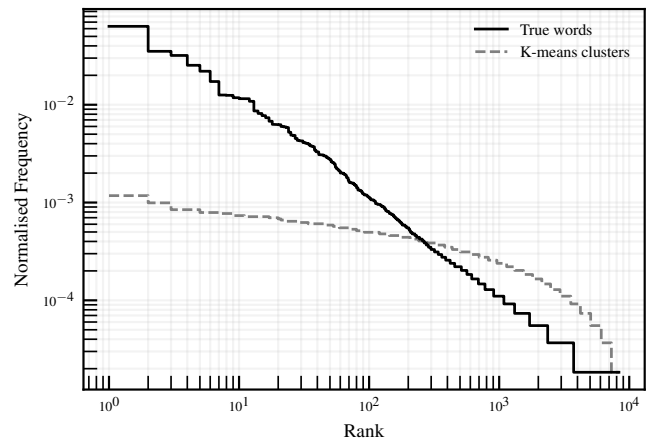


Fig. 1: True word-type distribution vs K-means cluster frequencies over true word segments using averaged WavLM features on LibriSpeech dev-clean. K-means produces a near-uniform distribution, failing to capture the long-tailed Zipfian structure of natural language.

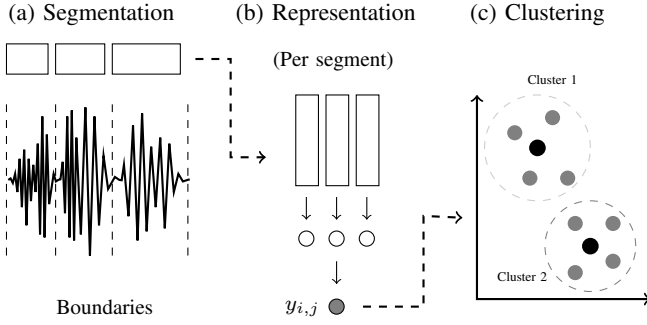


Fig. 2: The typical unsupervised term discovery pipeline: (a) speech audio is partitioned into segments; (b) each segment is mapped to a representation using an SSL model; and (c) a clustering algorithm groups segments into a lexicon of types.

modelling [28]–[30].

In all of the above, we use SSL embeddings trained on English data and perform experiments on English. To match a real zero-resource setting, we repeat our comparisons on Afrikaans and French data without altering the system settings. This allows us to determine whether our findings are consistent when moving to unseen languages.

Our contributions are threefold: (1) We show that the choice of clustering method plays a crucial role in unsupervised lexicon induction: bottom-up methods consistently outperform centre-based clustering across segmentation conditions and evaluation languages. (2) We analyse the distributional properties of induced lexicons and show that, compared to centre-based approaches like K-means, bottom-up methods better recover the long-tailed Zipfian distribution of word-level lexicons while also adapting to the less skewed distributions observed for syllabic units. (3) We show that graph clustering is a particularly useful bottom-up approach with two interpretable hyperparameters—a similarity threshold and a resolution parameter—that provide control over the induced type-frequency distribution.

Code is provided at: <https://github.com/adendorffy/zipf-clus>.

II. METHODOLOGY

We follow the three typical stages of unsupervised term discovery, as illustrated in Fig. 2, keeping the representation stage fixed across all experiments while varying the segmentation levels and clustering methods. This allows us to assess how different clustering assumptions and segmentation granularities affect lexicon quality and structure.

A. Segmentation and representation

Given a segmentation of the input speech by some unsupervised method (Fig. 2a), we define each segment $x_{i,j}$ by its start and end frames i and j . Each segment consists of $j - i$ feature frames from a self-supervised learning (SSL) model (Fig. 2b). After applying principal component analysis (PCA) to reduce the dimensionality, the SSL features are averaged across time to produce a single fixed-dimensional embedding $y_{i,j}$. Although simple, this averaging approach for obtaining segmental representations has been shown to be

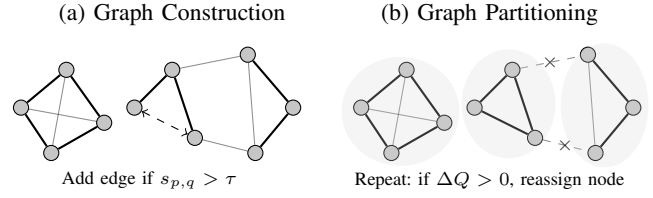


Fig. 3: Graph construction using similarity threshold τ and partitioning via the Leiden algorithm, controlled by resolution parameter γ .

highly effective [31]–[33]. Fixed-dimensional representations are also useful for the next step, clustering.

B. Clustering methods

The final step in the term discovery pipeline is to cluster segments to form a lexicon (Fig. 2c). This is the focus of this paper; we will use the same fixed-dimensional embeddings in all cases, and investigate the lexicons induced when altering the clustering method.¹ Altogether, we compare five clustering methods: two bottom-up methods that operate on pairwise relationships between segment embeddings, and three centre-based methods that represent clusters through centroids or parametric components. We describe the bottom-up methods first.

Graph clustering: This approach proceeds in the two stages seen in Fig. 3. First, a similarity graph $G = (V, E)$ is constructed (Fig. 3a). Each vertex $v \in V$ corresponds to a segment embedding $y_{i,j}$. For each pair of vertices v_p and v_q , the corresponding cosine similarity $s_{p,q}$ is computed. If $s_{p,q} > \tau$, an edge $e \in E$ is added between v_p and v_q with weight $s_{p,q}$. Higher values of this threshold parameter τ yield sparser graphs with smaller, more isolated communities, resulting in a flatter type-frequency distribution.

The connected components of this thresholded graph form an initial partition (Fig. 3a). However, this partition is coarse: segments may be grouped together through chains of pairwise similarities exceeding τ , which can result in communities that contain pairs of segments that are not similar. In the second stage, we apply the Leiden algorithm [34] to refine this partitioning (Fig. 3b). Leiden searches for a partition that locally maximises a quality function over the graph. For this, we employ the constant Potts model (CPM) [35]:

$$Q = \sum_{p=1}^N \sum_{q=1}^N (s_{p,q} - \gamma) \cdot \delta(c_p, c_q) \quad (1)$$

where N is the total number of vertices, c_p and c_q denote the community assignments for vertices p and q , and $\delta(c_p, c_q)$ is an indicator equalling 1 if the two vertices are assigned to the same community and 0 otherwise. The CPM objective rewards pairs of vertices assigned to the same community according

¹Using fixed-dimensional representations allows different clustering approaches to be compared on equal footing. But two of the methods (graph clustering and agglomerative clustering) operate on pairwise similarities and are therefore compatible with variable-length representations, such as discrete unit sequences [21]. We nevertheless restrict all clustering methods to the same fixed-dimensional embeddings for a fair comparison. In preliminary experiments, we did also evaluate graph clustering using discrete SSL units, but observed no performance improvements over fixed-dimensional embeddings.

to their edge weight $s_{p,q}$, while penalising such assignments by a fixed cost γ . The resolution parameter γ therefore gives another control over cluster granularity: higher values of γ favour smaller, more homogeneous clusters, while lower values allow larger, more inclusive clusters. Importantly, γ operates within the graph structure defined by τ : it can further subdivide communities but cannot recover connections excluded by τ .

We are not the first to propose graph clustering for unsupervised term discovery, but we are the first to use the modern Leiden partitioning with CPM. Park and Glass [8] used the older Newman algorithm [36] to partition the graph. Later work by Lyzinski et al. [9] applied the Louvain algorithm [37], a predecessor to Leiden. They also optimised modularity rather than CPM. We use Leiden with CPM because it is more robust to disconnected communities [34] and gives direct control over cluster granularity through γ . This is important because we want the resulting clusters to follow a particular distribution. In our own preliminary experiments, CPM also outperformed modularity when applied with the Leiden algorithm.

Agglomerative clustering: Our second bottom-up method builds clusters hierarchically through successive merges. At each step, the two clusters with the lowest average pairwise distance are joined, a criterion called average linkage [38], [39]. The distance between two clusters A and B is defined as:

$$d(A, B) = \frac{1}{|A||B|} \sum_{a \in A} \sum_{b \in B} d(y_a, y_b) \quad (2)$$

where $d(y_a, y_b)$ is the distance between embeddings y_a and y_b . Unlike Ward linkage [40], which minimises within-cluster variance and therefore favours compact clusters around centroids, average linkage operates over average pairwise distances. This does, however, make it more computationally expensive, as it requires maintaining and updating pairwise distances between clusters at each merge step. Rather than stopping merges at a distance threshold, we cut the hierarchy to obtain a desired number of clusters K . This is in contrast to graph clustering, where the parameters are adjusted to approximate the target K .

Centre-based methods: We compare against three centre-based methods that represent each cluster by a central point. K-means partitions data into K clusters by minimising within-cluster variance, implicitly favouring compact clusters around centroids. BIRCH [25] incrementally builds a tree of compact subclusters by absorbing each new point into the nearest subcluster within a fixed threshold; if no such subcluster exists, a new one is created. Since we provide a target K , BIRCH then applies agglomerative clustering to the subcluster centroids to obtain the final partition. This makes BIRCH highly efficient, but also constrains it toward compact clusters, since the initial subcluster construction is based on a fixed threshold around local centroids. The finite Bayesian Gaussian mixture model (FBGMM) [41], [42] is a Bayesian variant of a Gaussian mixture model (GMM) that places a Dirichlet prior over mixture weights, allowing greater variation in cluster sizes than a standard GMM. We include it as a representative probabilistic method and because of its prior use in unsupervised term

discovery [14]. Although distributions over cluster means are modelled, the FBGMM is still fundamentally centre-based.

Together, these five methods allow us to test whether bottom-up approaches, which build clusters from pairwise similarities, are better suited than centre-based methods for recovering the long-tailed type-frequency structure of natural lexicons.

III. EXPERIMENTAL SETUP

We evaluate the five clustering methods across three segmentation conditions (ground-truth words, ground-truth syllables, and unsupervised syllabic segments) and three languages (English, Afrikaans, and French). This allows us to analyse sensitivity to segmentation choice and to assess whether the observed clustering behaviour is consistent across languages.

Datasets: For English, we use LibriSpeech dev-clean [43], comprising 4.5 hours of active speech from 40 speakers. For Afrikaans, we use 2 hours of speech from five speakers drawn from FLEURS [44]. Afrikaans is a low-resource Germanic language (like English), spoken in southern Africa. For French, we use a 4.2-hour subset (comparable to the English set) from Track 2 of the ZeroSpeech challenge [45], covering 12 speakers. Although French is a high-resource language, we treat it here as unseen. All three sets are relatively small, but these sizes are reasonable since no model training is taking place (an existing SSL model is used; see below). Moreover, many of the clustering approaches are computationally expensive; our goal here is a fair comparison without limitations due to computational constraints (although efficiency is discussed later).

Representations: We follow the best setup from [46]. Frame-level representations are extracted from the 21st layer of WavLM Large [11]. Extracted features are mean- and variance-normalised and reduced to 350 dimensions using PCA. Segment-level embeddings are then obtained by mean pooling over time. The resulting segment embeddings are mean-centred across the dataset and l_2 -normalised. WavLM was trained on large amounts of English data (no Afrikaans or French).

Segmentation conditions: Our bigger goal is to improve *unsupervised* term discovery, but here our focus is specifically on the effects of the clustering step. Given the limited performance of unsupervised word segmentation systems, we therefore start with ground-truth word boundaries. Unsupervised systems for detecting syllabic boundaries have been argued to be much more reliable [13]; we therefore also consider both ground-truth and unsupervised syllabic boundaries. Word and phone alignments for English and Afrikaans are obtained using a forced aligner [47], while the French alignments are provided with the data. Syllable alignments for all three languages are derived from the phone alignments using rule-based methods. Unsupervised syllable boundaries are inferred using the ZeroSyl method [30], where boundaries are placed at prominent peaks in the smoothed l_2 -norm signal of the WavLM features.

Implementation: Graph clustering is implemented using `igraph` [48]. K-means clustering is performed using `FAISS` [49] with K-means++ initialisation [50]. BIRCH and agglomerative clustering are implemented using `scikit-learn` [51]. BIRCH is configured with default

hyperparameters (branching factor 50, threshold 0.5), and cluster centroids are extracted directly from the leaf nodes. For agglomerative clustering, we use Euclidean distance for $d(y_a, y_b)$ in (2). (This worked better than cosine.) The FBGMM follows [14], with a fixed spherical covariance $\sigma^2 \mathbf{I}$ for all clusters; we set $\sigma^2 = 0.1$ based on development experiments.

Automatically estimating the lexicon size K is an open research question that we do not address here. We therefore set a target K across all methods. For the ground-truth word or syllable segmentation conditions, K is set to the true number of reference types. For unsupervised syllabic units, we set $K = 5,000$ for all three languages. This is based on work in syllable-based spoken language modelling, where a fixed K of roughly this order is used across datasets and languages [28], [52], [53]. For methods that require a fixed number of clusters, the target K is specified directly. For graph clustering, the resolution parameter γ is tuned so that the desired K is roughly reached.

Prior to partitioning using γ (Fig. 3b), the initial graph needs to be constructed using the similarity threshold τ (Fig. 3a). We set τ based on English development experiments with true word and syllable boundaries. These experiments showed that lower thresholds perform better on word segments, while higher thresholds perform better on syllable segments. We therefore use $\tau = 0.3$ for words, which is the lowest value that remained computationally tractable, and $\tau = 0.55$ for syllables,² which is the highest value that still yields a connected graph.

Evaluation: Apart from distribution plots, we need quantitative metrics to compare approaches. The most widely used metric for lexicon evaluation is normalised edit distance (NED) [1]: Each discovered segment is mapped to the ground-truth phone sequence with which it overlaps most, and the phone edit distance is then calculated between all pairs in a cluster. The distances are then aggregated. NED has recently been revised to account for an unfair bias that over-weights larger clusters; we use this revised metric, the normalised edit similarity (NES) [54]. Both NED and NES only evaluate consistency within a cluster, ignoring how true types are spread over clusters. To measure this, we therefore also use the inverse NES (iNES), which measures how consistently instances of each ground-truth type are clustered into the same cluster [54].³ NES and iNES can be combined by taking the harmonic mean, giving F_1 NES. Higher is better for all metrics. We also report bitrate [55], the lower bound on the average number of bits required to encode the tokenised speech (lower is better).

IV. RESULTS

A. Centre-based vs bottom-up clustering on English

We begin with English experiments, comparing the five clustering methods across the three segmentation conditions, where the SSL features were trained on the target language.

²An exception is made for the ground-truth French syllables, where we use $\tau = 0.5$ to obtain a connected graph, as the higher threshold of $\tau = 0.55$ resulted in a graph that was too sparse to partition into the target K .

³In [54], these metrics are called weighted normalised edit similarity (WNES) and inverse WNES (iWNES) to explicitly distinguish it from the original NED.

Table I shows that graph and agglomerative clustering substantially outperform K-means, BIRCH and the FBGMM, especially when considering the recall-based iNES and combined F_1 NES metrics. At the word level, centre-based methods achieve high NES of 88-89% but low iNES of 26-34%, indicating fragmentation of true types. In comparison, agglomerative and graph clustering more than double iNES, yielding F_1 NES scores of 67% and 68% compared to 40% for K-means at the word level. The same pattern holds for the ground-truth and unsupervised syllable-level conditions; although overall performance drops under unsupervised segmentation, the relative advantage of bottom-up methods persists. Under all three segmentation conditions, agglomerative and graph clustering also achieve the lowest bitrate, indicating distributions that efficiently encode the data.

Fig. 4 shows that these improvements correspond to clear differences in induced lexicon structure. K-means, BIRCH, and the FBGMM produce flat type-frequency distributions, consistent with fragmentation of frequent types (as per iNES). In contrast, graph and agglomerative clustering recover distributions that more closely follow the true distributions. Comparing the gold ground-truth distributions (solid black lines), we see that the long tail of true words is less pronounced at the syllable level (as expected). Regardless, the bottom-up graph and agglomerative methods are able to track the true distribution for both word- and syllable-level lexicon discovery.

Graph vs agglomerative: Both bottom-up approaches provide a clear benefit, but there is no clear winner. Although computational efficiency is not our core focus here, it is worth noting that agglomerative is much slower because of its repeated pairwise comparisons: on an 8-core 3.60 GHz CPU machine, agglomerative takes roughly 9 minutes compared to graph clustering’s 3 minutes when clustering the true English word

TABLE I: Lexicon evaluation for English across three segmentation conditions. Bitrate is measured in bits/second.

Method	K	NES $\uparrow$	iNES $\uparrow$	F_1 NES $\uparrow$	Bitrate $\downarrow$
Word-level ($K = 8,372$)					
K-means	8,372	87.86	26.29	40.47	43.02
BIRCH	8,372	88.12	33.95	49.01	41.79
FBGMM	8,372	88.72	32.07	47.11	43.22
Agglomerative	8,372	91.31	53.31	67.32	38.50
Graph ($\tau = 0.3$)	8,372	90.41	54.84	68.43	38.18
Syllable-level ($K = 4,473$)					
K-means	4,473	79.12	20.62	32.72	56.20
BIRCH	4,473	78.82	23.55	36.27	55.57
FBGMM	4,473	78.07	23.28	35.86	55.74
Agglomerative	4,473	77.08	42.18	54.52	49.52
Graph ($\tau = 0.55$)	4,473	74.78	45.16	56.31	34.30
Unsupervised syllabic: ZeroSyl ($K = 5,000$)					
K-means	5,000	68.25	14.87	24.43	63.00
BIRCH	5,000	67.55	17.43	27.71	61.97
FBGMM	5,000	67.30	16.13	26.02	62.64
Agglomerative	5,000	64.87	29.74	40.79	55.34
Graph ($\tau = 0.55$)	4,999	62.51	32.18	42.49	53.22

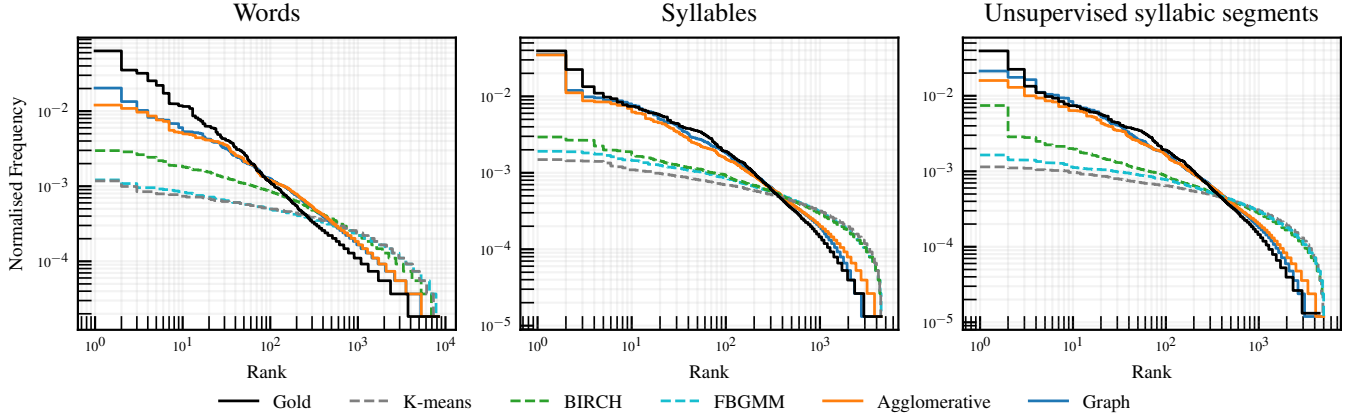


Fig. 4: Type-frequency distributions for induced lexicons on English across the three segmentation conditions. Dashed lines correspond to centre-based methods while solid lines indicate bottom-up clustering techniques. The black line shows the true word or syllable distribution.

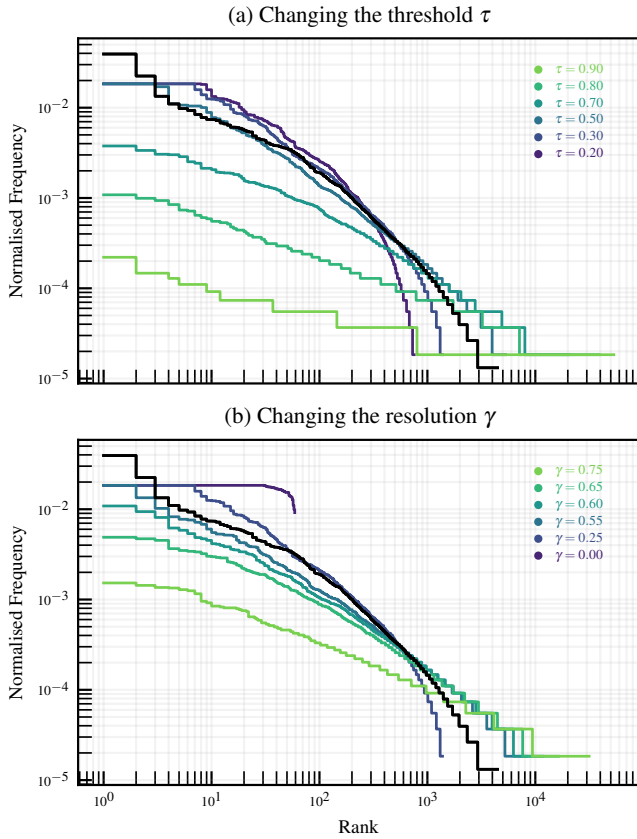


Fig. 5: Effect of the (a) threshold τ and (b) resolution γ graph clustering hyperparameters on the type-frequency distribution for true English word segments.

segments. Agglomerative clustering also has only a single setting: determining when to cut the tree, while graph clustering gives more explicit control over the resulting distributions through its two hyperparameters.

Distributions induced through graph clustering: Fig. 5 illustrates how the distributions change with the two graph clustering hyperparameters. Higher τ yields sparser graphs with

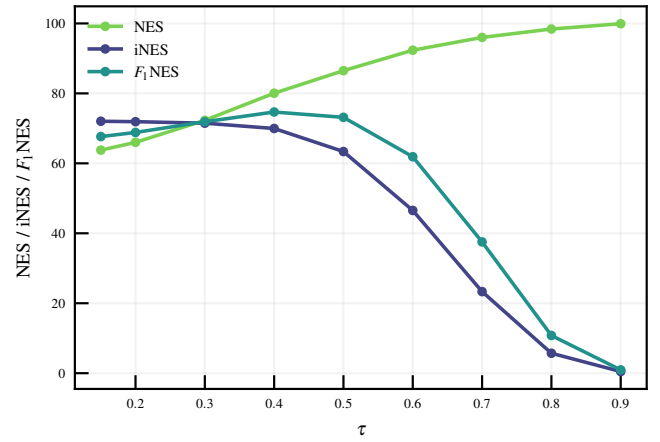


Fig. 6: NES, iNES, and F_1 NES at different thresholds τ for true English word segments.

flatter, more uniform distributions better suited to syllables; lower τ produces denser graphs whose distributions more closely follow the true word distribution (consistent with optimal τ settings from development; see Sec. III). The same trend holds for γ (Fig. 5b), but γ 's ability to approximate the true distribution is bounded by τ : a sufficiently high τ could produce a graph too sparse for γ to recover Zipfian structure. Fig. 6 further shows how τ governs a trade-off between NES and iNES. At high τ , the graph fragments into many small communities, driving up NES while iNES and F_1 NES collapse. At low τ , frequent types absorb neighbouring segments into large communities, improving iNES at the cost of NES. Together, τ and γ therefore provide interpretable control over lexicon structure that goes beyond simply fixing K .

Taken together, the English results show that the clustering method affects both lexicon quality and the induced type-frequency distribution, and that bottom-up methods like graph clustering outperform centre-based methods like K-means. The next section examines whether these advantages persist when applying English-trained SSL features on unseen languages.

TABLE II: Lexicon evaluation for Afrikaans across three segmentation conditions. Bitrate is measured in bits/second.

Method	K	NES $\uparrow$	iNES $\uparrow$	F_1 NES $\uparrow$	Bitrate $\downarrow$
Word-level ($K = 4,973$)					
K-means	4,973	67.86	14.75	24.23	33.22
Agglomerative	4,973	71.93	26.29	38.51	31.15
Graph ($\tau=0.3$)	4,971	70.50	24.81	36.71	30.88
Syllable-level ($K = 3,489$)					
K-means	3,489	66.11	13.85	22.91	48.44
Agglomerative	3,489	68.37	28.39	40.12	44.18
Graph ($\tau=0.55$)	3,496	59.26	41.72	48.97	37.08
Unsupervised syllabic: ZeroSyl ($K = 5,000$)					
K-means	5,000	61.24	10.05	17.27	64.49
Agglomerative	5,000	59.94	18.46	28.22	58.33
Graph ($\tau=0.55$)	5,001	56.67	20.83	30.46	57.25

B. Centre-based vs bottom-up clustering with English-trained SSL features on Afrikaans and French

Keeping the same setup as for English, we repeat the experiments on Afrikaans and French data while still using the English-trained SSL model (WavLM Large). Here we compare the bottom-up approaches only to K-means, since BIRCH and FBGM performed similarly to K-means on English.

Tables II and III show that the same pattern of results observed for English holds for both Afrikaans and French, with graph and agglomerative clustering outperforming the standard centre-based K-means approach. iNES scores increase substantially at the word level, from 15% to 25–26% on Afrikaans and from 10% to 20–22% on French. At the syllable level, graph clustering has a clearer advantage over agglomerative clustering, particularly for Afrikaans, where it achieves the highest iNES and F_1 NES at both syllable segmentations. For French, this advantage only holds for the ground-truth syllable segmentation. Overall, performance degrades relative to English in both languages—more severely for French than for Afrikaans. Fig. 7 shows word-level Zipf plots that mimic the results from the metrics: for both Afrikaans and French, graph and agglomerative clustering recover type-frequency distributions that more closely follow the true distribution than K-means.

V. CONCLUSION

This paper examined the role of clustering in unsupervised term discovery. We started by showing that the standard centre-based K-means approach has an inductive bias producing similarly sized clusters—a poor fit to the Zipfian distribution of real language. Across three segmentation conditions and three languages, bottom-up methods (graph and agglomerative) consistently improved type-level recovery over centre-based algorithms (K-means, BIRCH, finite Bayesian GMM). Graph clustering additionally provides interpretable control over the induced lexicon through two hyperparameters. Comparable performance from agglomerative clustering suggests that relaxing centre-based geometric assumptions, rather than the

TABLE III: Lexicon evaluation for French across three segmentation conditions. Bitrate is measured in bits/second.

Method	K	NES $\uparrow$	iNES $\uparrow$	F_1 NES $\uparrow$	Bitrate $\downarrow$
Word-level ($K = 7,776$)					
K-means	7,776	65.66	9.91	17.22	40.97
Agglomerative	7,776	65.32	22.17	33.10	36.36
Graph ($\tau=0.3$)	7,776	62.92	20.42	30.83	35.73
Syllable-level ($K = 2,522$)					
K-means	2,522	60.43	8.57	15.01	47.61
Agglomerative	2,522	52.24	21.32	30.28	38.79
Graph ($\tau=0.5$)	2,521	46.77	25.48	32.99	34.22
Unsupervised syllabic: ZeroSyl ($K = 5,000$)					
K-means	5,000	58.64	8.20	14.39	67.94
Agglomerative	5,000	50.99	19.85	28.58	56.44
Graph ($\tau=0.55$)	5,010	46.82	18.54	26.56	54.95

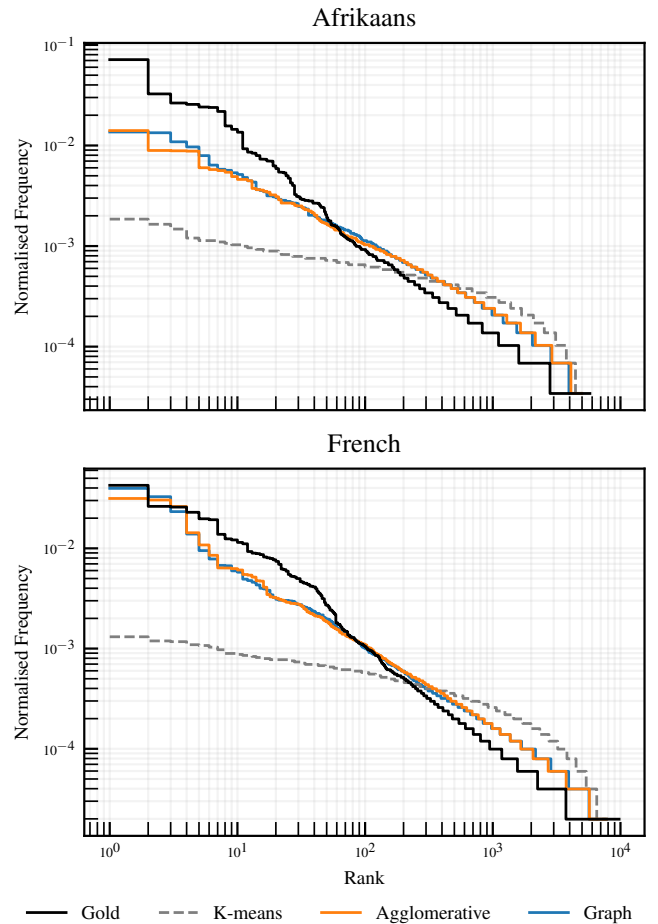


Fig. 7: Type-frequency distributions for induced lexicons on Afrikaans and French data using ground-truth word segments.

specific algorithm, is the key factor. These findings establish clustering as a central modelling decision in unsupervised term discovery. We found that graph clustering is computationally more efficient than agglomerative clustering, but both scale poorly. To be usable on realistically sized datasets, future work should focus on developing scalable bottom-up methods.

REFERENCES

- [1] B. Ludusan *et al.*, “Bridging the gap between speech technology and natural language processing: An evaluation toolbox for term discovery systems,” in *LREC*, 2014.
- [2] J. R. Saffran, R. N. Aslin, and E. L. Newport, “Statistical learning by 8-month-old infants,” *Science*, 1996.
- [3] P. W. Jusczyk, “Some critical developments in acquiring native language sound organization during the first year,” *The Annals of Otology, Rhinology and Laryngology*, 2002.
- [4] E. Bergelson and D. Swingley, “At 6-9 months, human infants know the meanings of many common nouns,” *Proceedings of the National Academy of Science*, 2012.
- [5] O. Räsänen, “Computational modeling of phonetic and lexical learning in early language acquisition: Existing models and future directions,” *Speech Communication*, 2012.
- [6] E. Dupoux, “Cognitive science in the era of artificial intelligence: A roadmap for reverse-engineering the infant language-learner,” *Cognition*, 2018.
- [7] L. Besacier, E. Barnard, A. Karpov, and T. Schultz, “Automatic speech recognition for under-resourced languages: A survey,” *Speech Communication*, 2014.
- [8] A. S. Park and J. R. Glass, “Unsupervised pattern discovery in speech,” *IEEE Transactions on Audio, Speech and Language Processing*, 2008.
- [9] V. Lyzinski, G. Sell, and A. Jansen, “An evaluation of graph clustering methods for unsupervised term discovery,” in *Interspeech*, 2015.
- [10] W. N. Hsu *et al.*, “HuBERT: Self-supervised speech representation learning by masked prediction of hidden units,” *IEEE/ACM Transactions on Audio Speech and Language Processing*, 2021.
- [11] S. Chen *et al.*, “WavLM: Large-scale self-supervised pre-training for full stack speech processing,” *IEEE Journal on Selected Topics in Signal Processing*, 2022.
- [12] C.-y. Lee, T. J. O’Donnell, and J. Glass, “Unsupervised lexicon discovery from acoustic input,” *Transactions of the Association for Computational Linguistics*, 2015.
- [13] O. Räsänen, G. Doyle, and M. C. Frank, “Unsupervised word discovery from speech using automatic segmentation into syllable-like units,” in *Interspeech*, 2015.
- [14] H. Kamper, K. Livescu, and S. Goldwater, “An embedded segmental k-means model for unsupervised segmentation and clustering of speech,” in *ASRU*, 2017.
- [15] S. Bhati, J. Villalba, P. Želasko, and N. Dehak, “Self-expressing autoencoders for unsupervised spoken term discovery,” in *Interspeech*, 2020.
- [16] S. Bhati *et al.*, “Segmental contrastive predictive coding for unsupervised word segmentation,” in *Interspeech*, 2021.
- [17] R. Algayres *et al.*, “DP-Parser: Finding word boundaries from raw speech with an instance lexicon,” *Transactions of the Association for Computational Linguistics*, 2022.
- [18] S. Cuervo *et al.*, “Contrastive prediction strategies for unsupervised segmentation and categorization of phonemes and word,” in *ICASSP*, 2022.
- [19] Y. Okuda, R. Ozaki, S. Komura, and T. Taniguchi, “Double articulation analyzer with prosody for unsupervised word and phone discovery,” *IEEE Transactions on Cognitive and Developmental Systems*, 2023.
- [20] H. Kamper, “Word segmentation on discovered phone units with dynamic programming and self-supervised scoring,” *IEEE/ACM Transactions on Audio Speech and Language Processing*, 2023.
- [21] B. van Niekerk, J. Zaidi, M.-A. Carbonneau, and H. Kamper, “Spoken-term discovery using discrete speech units,” in *Interspeech*, 2024.
- [22] A. Pasad, C.-M. Chien, S. Settle, and K. Livescu, “What do self-supervised speech models know about words?,” *Transactions of the Association for Computational Linguistics*, 2024.
- [23] S. Malan, B. van Niekerk, and H. Kamper, “Unsupervised word discovery: Boundary detection with clustering vs. dynamic programming,” in *ICASSP*, 2025.
- [24] S. T. Piantadosi, “Zipf’s word frequency law in natural language: a critical review and future directions,” *Psychonomic Bulletin & Review*, 2014.
- [25] T. Zhang, R. Ramakrishnan, and M. Livny, “BIRCH: An efficient data clustering method for very large databases,” *ACM SIGMOD Record*, 1996.
- [26] J. A. Bilmes, “A gentle tutorial of the EM algorithm and its application to parameter estimation for Gaussian mixture and hidden Markov models,” tech. rep., International Computer Science Institute, 1998.
- [27] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [28] C. J. Cho *et al.*, “Sylber: Syllabic embedding representation of speech from raw audio,” in *ICLR*, 2025.
- [29] A. Baade, P. Peng, and D. Harwath, “SyllableLM: Learning coarse semantic units for speech language models,” in *ICLR*, 2025.
- [30] N. Visser, S. Malan, D. Slabbert, and H. Kamper, “ZeroSyl: Simple zero-resource syllable tokenization for spoken language modeling,” in *Interspeech*, 2026.
- [31] K. Levin, K. Henry, A. Jansen, and K. Livescu, “Fixed-dimensional acoustic embeddings of variable-length segments in low-resource settings,” in *ASRU*, 2013.
- [32] C. Jacobs and H. Kamper, “Leveraging multilingual transfer for unsupervised semantic acoustic word embeddings,” *IEEE Signal Processing Letters*, 2024.
- [33] J. Herreillers, C. Jacobs, and T. Niesler, “Low-resource keyword spotting using contrastively trained transformer acoustic word embeddings,” *arXiv preprint arXiv:2506.17690*, 2025.
- [34] V. A. Traag, L. Waltman, and N. J. van Eck, “From Louvain to Leiden: Guaranteeing well-connected communities,” *Scientific Reports*, 2019.
- [35] V. A. Traag, P. V. Dooren, and Y. Nesterov, “Narrow scope for resolution-limit-free community detection,” *Physical Review E*, 2011.
- [36] M. E. J. Newman, “Fast algorithm for detecting community structure in networks,” *Physical Review E*, 2004.
- [37] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, “Fast unfolding of communities in large networks,” *Journal of Statistical Mechanics: Theory and Experiment*, 2008.
- [38] S. Johnson, “Hierarchical clustering schemes,” *Psychometrika* 32, 1967.
- [39] F. Nielsen, *Introduction to HPC with MPI for Data Science*, ch. Hierarchical Clustering, pp. 195–211. Springer, 2016.
- [40] F. Murtagh and P. Legendre, “Ward’s hierarchical agglomerative clustering method: Which algorithms implement Ward’s criterion?,” *Journal of Classification*, 2014.
- [41] C. E. Rasmussen, “The infinite Gaussian mixture model,” in *NIPS*, 1999.
- [42] H. Kamper, A. Jansen, and S. Goldwater, “Unsupervised lexical clustering of speech segments using fixed-dimensional acoustic embeddings,” in *SLT*, 2014.
- [43] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, “LibriSpeech: An ASR corpus based on public domain audio books,” in *ICASSP*, 2015.
- [44] A. Conneau *et al.*, “FLEURS: Few-shot learning evaluation of universal representations of speech,” in *SLT*, 2023.
- [45] E. Dunbar *et al.*, “The Zero Resource Speech Challenge 2017,” in *ASRU*, 2017.
- [46] D. Slabbert, S. Malan, and H. Kamper, “Unsupervised lexicon learning from speech is limited by representations rather than clustering,” in *ICASSP*, 2026.
- [47] M. McAuliffe *et al.*, “Montreal forced aligner: Trainable text-speech alignment using kald,” in *Interspeech*, 2017.
- [48] G. Csardi and T. Nepusz, “The igraph software package for complex network research,” *InterJournal Complex Systems*, 2006.
- [49] M. Douze *et al.*, “The Faiss library,” *IEEE Transactions on Big Data*, 2026.
- [50] D. Arthur and S. Vassilvitskii, “K-means++: The advantages of careful seeding,” in *SODA*, 2007.
- [51] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, 2011.
- [52] P. Peng, S.-W. Li, O. Räsänen, A. Mohamed, and D. Harwath, “Syllable discovery and cross-lingual generalization in a visually grounded, self-supervised speech model,” in *Interspeech*, 2023.
- [53] C. J. Cho *et al.*, “SD-HuBERT: Sentence-level self-distillation induces syllabic organization in HuBERT,” in *ICASSP*, 2024.
- [54] S. Malan, D. Slabbert, and H. Kamper, “Revisiting lexicon evaluation in unsupervised word discovery,” *arXiv preprint arXiv:2606.06183*, 2026.
- [55] E. Dunbar *et al.*, “The Zero Resource Speech Challenge 2020: Discovering discrete subword and word units,” in *Interspeech*, 2020.